Decoupled Information-Set MCTS with a Card-Value Model for Sushi Go!

Halit Ozger
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 2]
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 3]
Student no. [to be completed]
Queen Mary University of London
London, UK

***Abstract*—In Sushi Go! players pick cards simultaneously from hidden hands that rotate around the table, which basic Monte Carlo tree search (MCTS) does not model. Starting from TAG's BasicMCTS, we added a model of the game's scoring that drives evaluation and rollouts, rollouts that end with the round, max-n backups, decoupled UCT for the simultaneous choices and information-set search. On held-out deals the agent wins 52.6% of 500 three-player games at 100 ms against TAG's ISMCTS and our previous agent (chance 33.3%), [pending]% with four players (chance 25%) and [pending]% at 900 ms. Ablations show that the heuristic and the simultaneous-move tree matter most, that the tree's advantage tends to grow with the time budget, and that four players need different settings.**

Keywords*—*Monte Carlo tree search, information sets, simultaneous moves, card games, Sushi Go!, TAG

I. Introduction

Tabletop games combine chance, hidden information and several interacting players within small, well-defined rule sets, which makes them a long-standing test bed for game AI; the Tabletop Games framework (TAG) [1] gives many of them a common interface so that agents can be compared. Monte Carlo tree search (MCTS) [2] is the usual starting point for such agents, but its basic form assumes that the state is fully observed and that players move one after another [3]. *Sushi Go!*, one of TAG's games, breaks both assumptions: every player secretly picks one card from its hand, all players reveal their cards together and pass the rest of their hands on, and after three rounds the highest score wins. Klang et al. [4] introduced the game to TAG and assessed simultaneous action selection and complete information in it; the current version, which we use, shows an agent only the cards it has seen.

Four properties make the game hard for a search agent. (i) The opponents' hands are hidden until they are passed on. (ii) Moves are simultaneous: no card can be chosen in reaction to an opponent's card of the same turn. (iii) Many points are delayed and conditional – sets pay out only when completed, Maki rolls at the end of the round and Pudding at the end of the game – so short simulations miss them and long ones are noisy. (iv) Hands of nine (three players) or eight cards (four players) give up to 729 or 4 096 joint moves per turn, with 0.1–1 s per decision. TAG's BasicMCTS addresses none of the first three: it searches the one shuffled sample of the hidden cards it receives as if it were the true state, treats the players of a turn as moving one after another and evaluates random rollouts by the player's own score.

We ask which adaptations of MCTS to these properties make an agent stronger (RQ1), and how their value changes with the time per decision (RQ2) and the number of players (RQ3). We extend BasicMCTS in steps, keeping every modification as a switch, and test each in experiments of (mostly) 500 games. Our contributions are:

- a model of Sushi Go!'s scoring that estimates every player's final score and every card's marginal value, used both to evaluate states and as the rollout policy of all players;
- a re-implementation on BasicMCTS of round-end rollouts, max-n backups, decoupled UCT and information-set search, combined with this model and compared with TAG's own decoupled information-set MCTS;
- evidence that the best settings differ between three and four players;
- a study of 159 runs (67 850 games) with leave-one-out ablations, a 2^4 factorial design, validation on held-out deals and negative results that locate the agent's remaining limits.

II. Background

A. TAG and Sushi Go!

TAG [1] splits a game into a state, a forward model and parameters; an agent receives a copy of the state, in which every card it has not seen is reshuffled, and uses the forward model to simulate moves. In TAG's Sushi Go! the 108-card deck holds Tempura (5 points per pair), Sashimi (10 per triple), Dumplings (1, 3, 6, 10 or 15 points for one to five), Salmon, Squid and Egg Nigiri (2, 3 and 1 points, tripled on an earlier Wasabi), Maki rolls with one to three icons (6 and 3 points for the most and second-most icons of a round), Chopsticks (two cards in a later turn) and Pudding (+6 for the most and, with three or more players, −6 for the fewest at the end of the game). Each round deals $12 - n$ cards to each of n players. As the hands rotate, every hand of a round has been seen after $n - 1$ passes, so hidden information is concentrated at the start of each round, while the value of a card depends on what the others will take from the hands we pass on.

B. Monte Carlo Tree Search

MCTS [2] builds a search tree by repeating selection, expansion, simulation (rollout) and backpropagation. Upper Confidence bounds applied to Trees (UCT) [5] selects the child that maximises $Q + K\sqrt{(\ln N / n)}$, where Q is the child's mean reward, n its visits and N those of its parent. Domain knowledge can enter through heuristic rollout policies [2] or priors such as progressive bias [6]. A stronger rollout policy does not guarantee better estimates, however: Silver and Tesauro [7] showed that rollouts should be balanced, i.e., give unbiased value estimates, rather than be played by a strong policy.

All sections, tables and figures of this paper were drafted with Claude [15]; the prompts are given in the statement on the use of generative AI at the end of the paper.

C. Hidden Information, Simultaneous Moves and Several Players

Determinisation, or perfect-information Monte Carlo (PIMC) sampling, samples the hidden information, searches each sample as if it were the true state and combines the results. Long et al. [8] related its success in card games to properties such as how quickly hidden information is revealed during play, which in Sushi Go! happens within $n - 1$ turns. Its known flaws are strategy fusion – the search assumes that it can act differently in states it cannot tell apart – and non-locality [9]. Information-set MCTS (ISMCTS) [3] avoids strategy fusion by searching one tree over the searcher's information sets and sampling the hidden information anew in every iteration; it was developed for card games such as the three-player Dou Di Zhu. In its single-observer form the opponents' choices in the tree are not conditioned on their own private information; the multiple-observer form models it with one tree per player at a higher cost [3], an idea TAG's advanced MCTS agent offers as MultiTree MCTS [10].

Simultaneous moves are usually handled by decoupled UCT (DUCT): every player chooses its own action at a node from its own bandit statistics and the joint action leads to a child [11]. UCB-based DUCT need not converge to an equilibrium, so Lanctot et al. [11] compared it with EXP3 and regret-matching selection, and Tak et al. [12] found that the best variant depends on the game; both studies used two-player games such as Goofspiel. In multi-player games, UCT with max-n backups, in which every player maximises its own value, computes a mixed-strategy equilibrium and played as well as or better than earlier multi-player search algorithms in Sturtevant's experiments [13].

TAG's advanced MCTS agent already offers information sets, one tree per player and a decoupled option, but the assignment does not allow it as a basis; BasicMCTS has none of these ideas. How they combine with domain knowledge in a multi-player drafting game, what each contributes, and how this depends on the time budget and the number of players has not been studied: the earlier Sushi Go! study [4] predates the version with hidden hands. These are the questions this paper answers.

III. Method

Our agent, SushiMCTS, is a modified copy of TAG's BasicMCTS (package *players.basicMCTS*) and uses no code of TAG's advanced MCTS agent. Every modification is a switch in its parameter class (Table I), so all variants run the same code; Fig. 1 shows one iteration of the final agent. The search techniques come from the literature; our own elements are the card-value model shared by the evaluation and every player's simulation, and its integration with these techniques on BasicMCTS.

TABLE I. Modifications of BasicMCTS (✓ = used in the final agent with 3 / 4 players)

Id	Modification	3 p / 4 p
H	heuristic (2)–(3) from the card-value model (1)	✓ / ✓
B	ε-greedy card-value rollouts for every player	✓ / ✓
R	rollouts end when the round is scored	✓ / ✓
P	one branch per distinct card	✓ / ✓
M	max-n backups [13]	✓ / ✓
S	decoupled UCT, one node per turn [11]	✓ / ✓
I	single-observer information-set search [3]	✓ / ✓
X0	card-model constants tuned for 4 players [a]	– / ✓
W2	win-gap value for 4 players (Section III-D)	– / ✓ [b]
F, L	fast simulator, 8 rollouts per leaf	– / ✓ [b]
A	progressive bias [6]	dropped in v3
D	ensemble of up to 4 determinised trees [8]	replaced by I

[a] σ 15, *w* 0.7, α 0.5, *T* 6; Sashimi discount 0.5; Maki horizon 3 with 0.25 per icon; next-Dumpling weight 1.5; Pudding factor 1.25; average Nigiri 3; 3 cards per missing card left in the hands. [b] at 500 ms or more per decision.

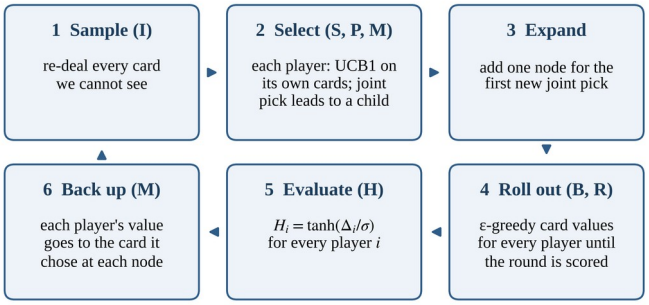


Fig. 1. One iteration of the final agent. The letters refer to Table I; after the time budget the agent plays its most visited card at the root.

A. Card-Value Model and Heuristic (H)

For a state s and player p the model estimates the final score

$$\hat{S}_p = S_p + \alpha U_p + M_p + w_t P_p, \quad (1)$$

where S_p is the banked score (TAG scores Tempura, Sashimi, Dumplings and Nigiri when they are revealed), U_p the value of unfinished sets, each weighted by a completion factor $\min(1, (r - m + 1)/T)$ (0 if $r < m$) for a set that needs m more cards with r picks left, M_p the Maki points p would get if the round ended now, and P_p the Pudding points if the game ended now, weighted by $w_t = t/3$ in round t ($\alpha = 0.5$, $T = 4$). The value of s for player i is

$$H_i(s) = \tanh(\Delta_i / \sigma), \quad (2)$$

$$\Delta_i = w(\hat{S}_i - \max_{j \neq i} \hat{S}_j) + (1 - w)(\hat{S}_i - \text{mean}_{j \neq i} \hat{S}_j), \quad (3)$$

with $\sigma = 15$ and $w = 0.7$: the first term rewards beating the leader, since only first place wins, and the second keeps a gradient when the agent is far ahead or behind. The marginal value $mv_p(c)$ of a card c is the change of $\hat{S}_p$ when p plays it.

B. Simulation (B, R)

B: every player plays its card with the highest *mv*, or a random card with probability $\varepsilon = 0.2$ (with Chopsticks, the second card is the best remaining one); the same policy models the opponents. **R:** a rollout stops when the current round has been scored; only Pudding carries over, which (1) captures, so the leaves are valued by banked points and the Pudding standing.

C. Tree (P, M, S, I)

P: identical cards lead to identical states, so a node has one branch per distinct card. **M:** a node stores one value per player and every player maximises its own value [13]. **S:** one node represents a whole turn; at a node every player p picks a card by

$$a_p = \text{argmax}_a \left[Q_p(a) + K\sqrt{(\ln N_p(a) / n_p(a))} \right] \quad (4)$$

from its own statistics, trying untried cards first; the joint choice is played in the forward model and leads to the child of that joint action [11]. In v3 the players of a turn appeared one after another in the tree, so a later player "saw" the card an earlier player had just chosen, which is impossible in the real game. **I:** every iteration starts from a new sample of the cards we cannot see and walks one shared open-loop tree, whose nodes store statistics but no states [3]. Statistics are keyed by card type; $n_p(a)$ counts how often p chose a at the node and the availability count $N_p(a)$ how many iterations reached the node with a in p's sampled hand. I replaces the determinisation ensemble D of v2 and v3, which split the budget between up to four independent trees. With three players $K = 0.5$.

D. Four Players

With four players the agent switches parameters (Section IV-D). It uses the card constants X0 (Table I), found by a search-free coordinate tuner that plays greedy card-value policies against each other. From 500 ms it also uses **W2** – $w = 1$; finished games valued by $\tanh((S_i - \max_{j \neq i} S_j)/2)$ with Pudding as tie-break; σ multiplied by $(R/2)^{0.5}$ for R rounds left; opponents in the backup valuing their lead over the others' mean – and **F**, TAG's rules re-implemented on integer arrays and checked move by move against the forward model, whose speed is spent on **L** = 8 rollouts per leaf, with $K = 1$. At 500 ms this gives 36 806 iterations of 8 rollouts per decision against 10 170 single-rollout iterations without F and L.

E. Experimental Setup

TABLE II. EXPERIMENTAL SETUP

Item	Setting
Framework	TAG as distributed with the module, Sushi Go! with its default rules; OpenJDK 21
Opponents	v2 (frozen copy of our previous agent): H, progressive bias, B, D, K 1.41. TAG ISMCTS: TAG's SushiGo_3+P_HiBudget configuration (information sets, one tree per player [10], UCB-Tuned, K 1, random rollouts to the game end, score heuristic). TAG DUCT: TAG's SushiGoDUCT configuration (decoupled UCT, K 1, information sets, lead heuristic, 30-move rollouts). BasicMCTS (K 1.41, 10-move random rollouts), Rolling Horizon Evolutionary Algorithm (RHEA; horizon 10, population 10), One-Step Look-Ahead (OSLA, score heuristic)
Budget	CPU time per decision, equal for all agents: 100 ms (most development), 500 and 900 ms
Seating	our agent in seat g mod n in game g; the other seats alternate between the two opponents (4 players: 2 + 1, swapped every 4 games)
Deals	game g uses seed $s + g$; since v4 a fixed deck order is shuffled with it, so equal s gives equal deals (paired runs); earlier runs used TAG's own deal, which differs between runs (independent samples)
Seeds	development s = 1, 2 (v3 also 1001–3001); ablation and last study s = 7; held-out validation s = 100 000, disjoint from all other deals
Games	500 per condition; 250 at 900 ms, in 4-player 500 ms development and in the 500/900 ms validation (48–73 s of CPU time per game); 300 in one run
Hardware	18-core laptop (6–16 games in parallel); last study and validation on a 4-core machine (4 in parallel)

Each game is an independent trial whose outcome for our agent is 1 for a win, $1/k$ for a k-way shared first place and 0 otherwise. Win rates are reported with Wilson 95% intervals [14]; the score margin (our score minus the best opponent's) as mean ± 1.96 standard errors (SE); differences between runs as the mean of the per-game differences on the same deals ± 1.96 SE (for independent samples, the difference of the means with the two-sample SE) and z = difference/SE, in percentage points (pp) for win rates. With 500 games a win rate near 50% has a 95% interval of ±4.4 pp, and a difference between two runs about ±5.7 pp even when paired, because the outcomes of two agents on the same deal correlate only weakly (0.0–0.15); the score margin is about twice as sensitive and served to screen ideas. We report unadjusted intervals and state where a conclusion survives a Holm correction. We expected (H1) every component to improve both measures, (H2) the decoupled information-set tree to matter more at longer budgets and (H3) the best settings to depend on the number of players.

IV. RESULTS

A. Development from v2 to v4 (RQ1, RQ2)

Fig. 2 traces the development with three players (independent samples). Removing progressive bias from v2 and adding P, R, M and $K = 0.5$ doubled the win rate at 100 ms (24.8% → 50.1%, +25.3 ± 5.8 pp). In a 2^4 factorial design on top of H + B + D + P (16 cells × 500 games) the main effects were +8.7 ± 2.1 pp for R (z = 8.1), +5.2 ± 2.2 for K = 0.5 instead of 1.41, +4.1 ± 2.0 for M and +3.6 ± 2.0 for a completion horizon T = 8; max-n backups needed less exploration (M × K +3.2 ± 2.2), and T = 8 was dropped because it overlaps with R (R × T −4.2 ± 2.1). Progressive bias no longer helped once informed rollouts were present (−1.2 ± 4.3 pp), and making the opponents in the tree follow the greedy card-value policy cost 16.9 ± 5.8 pp.

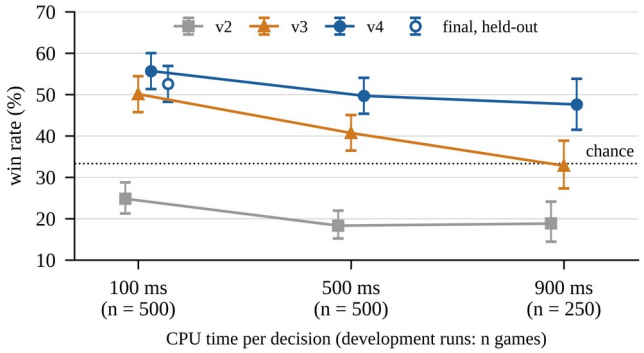


Fig. 2. Win rate with three players against v2 and TAG's ISMCTS by CPU time per decision. Lines: development runs (seed 1, independent samples; v2 = H + A + B + D, v3 = H + B + D + P + R + M, v4 = v3 + S + I − D). Open circles: the final agent on held-out deals. Bars: Wilson 95% intervals.

With longer budgets v3 lost 17.3 ± 7.3 pp from 100 to 900 ms but v4 only 8.1 ± 7.6 pp, so the gain from S and I rose from +5.6 ± 6.2 to +14.8 ± 8.5 pp; the rise (+9.2 ± 10.5 pp, z = 1.7) agrees with H2 but is not significant. S alone added +1.3 pp to v3 at 100 ms and +6.2 pp at 500 ms, so the decoupled tree carries most of the budget dependence. We attribute this to v3's sequential tree, in which later players react to cards chosen earlier in the same turn: a wrong model that more iterations exploit more. It is not a lack of search: at 900 ms v3 ran 19 696 iterations per decision, v4 21 246. Part of every agent's decline comes from the opponents: the TAG ISMCTS seats won 27.7% of v4's validation games at 100 ms but 35.1% at 500 ms.

B. Final Agent on Held-Out Deals (RQ1)

TABLE III. FINAL AGENT ON HELD-OUT DEALS

Setting	Opponents	*n*	Win % [95% CI]	Margin
3p/100	v2, ISMCTS	500	52.6 [48.2, 56.9]	+0.73 ± 0.46
3p/500	v2, ISMCTS	250	pending	pending
3p/900	v2, ISMCTS	250	pending	pending
4p/100	v2, ISMCTS	500	pending	pending
4p/500	v2, ISMCTS	250	pending	pending
3p/100	DUCT, ISMCTS	500	pending	pending
3p/100	ISMCTS × 2 [a]	300	54.2 [48.5, 59.7]	+1.66 ± 0.69
3p/100	BasicMCTS, RHEA [a]	500	84.2 [80.7, 87.1]	+6.68 ± 0.60
3p/100	OSLA, BasicMCTS [a]	500	87.2 [84.0, 89.8]	+9.74 ± 0.75

Setting: players / ms per decision. Seed block 100 000 except [a] seed block 7 (used for no decision about these opponents). Margin: our score minus the best opponent's, ± 95% CI. Chance: 33.3% (3 players), 25% (4 players).

Table III reports the final agent on held-out deals. Against v2 and TAG's ISMCTS it wins 52.6% at 100 ms with three players. On seed block 7, which overlaps the development deals, it had won 55.0% at 100 ms (−2.4 ± 6.2 pp), so we see no sign that the development overfitted its deals. BasicMCTS, RHEA and OSLA win at most 12.8% of their seats; OSLA wins none, as cards are revealed only when all players have chosen, so its look-ahead rarely sees a change of score.

C. Ablation (RQ1)

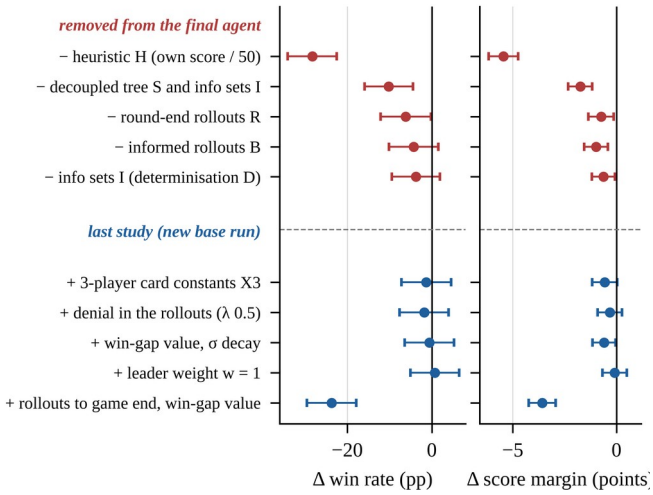


Fig. 3. Paired effects with three players at 100 ms (seed 7, 500 games each), ± 95% CI. Top: one component removed from the final agent (base: final agent, 55.0%). Bottom: changes tested in our last study, against a re-run of the final agent on the 4-core machine (58.1%).

Fig. 3 (top) removes one component at a time. Every removal lowers both measures; on the score margin all five differences survive a Holm correction, on the win rate only those of H and of S + I (−R: $z = 2.0$, −B: 1.4, −I: 1.3), so H1 holds for the margin and only partly for wins. Without H the leaves are valued by TAG's default heuristic, the own score divided by 50, which ignores the opponents' scores – although only first place wins – and the Pudding standing, and does not match the scale of K: this costs 28.3 ± 5.8 pp. S and I together are worth 10.2 ± 5.7 pp ($z = 3.5$) but I alone only 3.8 ± 5.7 pp, as Long et al.'s analysis [8] suggests for a game that reveals its hidden information within $n − 1$ turns. P and M cannot be removed from the final agent, since decoupled selection needs one value per player and the card-type statistics of I merge identical cards. The components also change the style of play: at 500 ms v3 scored fewer points than v2 from Sashimi (−2.7 per game) and Maki (−1.9) but more from Dumplings (+1.9) and Pudding (+1.3), sets that cannot fail or are decided over the whole game.

D. Four Players and the Limits of Search (RQ3)

With four players the 3-player configuration won 46.4% at 100 ms and 38.4% at 500 ms (development; chance 25%). None of the 4-player changes is individually significant: X0 added +3.2 ± 6.2 pp at 100 ms; at 500 ms X0 with W2 and $K = 1$ added +7.2 ± 8.6 pp and F + L a further +1.0 ± 7.9 pp, whereas F + L cost 6.6 ± 6.0 pp at 100 ms ($z = 2.2$). Together they support H3: X0 and W2 cost the 3-player agent 7.7 ± 5.4 pp ($z = 2.8$) at 500 ms, so their effect differs by 14.9 ± 10.2 pp ($z = 2.9$) between the table sizes. At 500 ms a 4-player tree cut to depth 1 lost 17.4 ± 8.2 pp ($z = 4.2$) and our agent limited to 100 ms 9.0 ± 7.7 pp ($z = 2.3$), yet 1 000 ms did not help (−4.8 ± 8.1 pp). On 73 positions from random four-player games the 500 ms tree has on average 704 joint-move children at the root, 53% of them visited once, and a mean depth of 1.5; doubling the time or the rollouts per leaf changed the chosen card in 2.7% and 5.5% of the positions, no more often than re-running the search (5.5%).

E. What Did Not Help

The last study (Fig. 3, bottom) changed the model and the value. Card constants tuned for three players (X3) make a far stronger greedy policy – it wins 45.0% of greedy games against two X0 players – yet did not strengthen the search (−1.3 ± 5.8 pp), consistent with simulation balancing [7]. Denial ("hate drafting": simulated players also value taking the next player's best card) gained about 1 pp in greedy play and nothing in the search (−1.8 ± 5.8 pp); a win-gap or leader-only value did not change the win rate (−0.6, +0.7 pp), and rollouts to the game end with a win-gap value cost 23.7 ± 5.8 pp ($z = 8.0$).

V. Conclusions and Future Work

We adapted TAG's BasicMCTS to Sushi Go! with a card-value model for evaluation and simulation, round-end rollouts, max-n backups, decoupled UCT and information-set search. For RQ1, the heuristic (28.3 pp when removed) and the decoupled information-set tree (10.2 pp) contribute most, and round-end rollouts led v3's factorial (+8.7 pp). On held-out deals the final agent wins 52.6% of three-player games at 100 ms against TAG's ISMCTS and our previous agent. For RQ2, the decoupled tree loses less strength than the sequential one at longer budgets (−8.1 against −17.3 pp from 100 to 900 ms), though not significantly. For RQ3, the best settings differ between three and four players ($z = 2.9$), and with four players the wide joint-move tree, not the time, limits the search.

The study has limits: our opponents are framework agents and our own previous version; the long-budget validation and the 4-player 500 ms development rest on 250 games (about ±6 pp); the factorial and budget studies used independent rather than paired deals; and CPU-time budgets hide wall-clock effects such as garbage collection. As hand-tuned changes to the model no longer move the agent, future work should learn the evaluation from game outcomes, e.g. by regressing final ranks on features of round-end states. Regret matching or EXP3 could replace UCB in the decoupled selection [11], as the best variant depends on the game [12], and opponent models fitted to the observed picks of real opponents could replace the shared greedy policy, which models all players alike.

Use of Generative AI

We used Claude [15] (Anthropic), through the Claude Code tool, throughout the project: to write and modify the Java code of our agent on top of BasicMCTS, to write and

run the experiment and statistics scripts, and to draft all sections, tables and figures of this paper from the logs of our runs; every number comes from the result files in our repository (folders experiments_v3 to experiments_v5). The authors take full responsibility for the code and the paper, including any errors. The paper was generated with the prompt (original Turkish, then English translation): *"simdi reponun icindeki butun herseyi incele ve hocaya direk yuklyecegimiz raporuda olusutr repoda ornekler tipler brief hersey var 100 alcagimiz bir rapor olsun her raporu tamaladiginda baska bir agent spawn edip bagimsiz durusstce kontrol taki yuzde yuz alacagana emin olankadar looptan cikma"* – "Now examine everything in the repository and also create the report we will upload directly to the lecturer; the repository has examples, tips, the brief, everything. It should be a report that gets 100. Every time you finish the report, spawn another agent to check it independently and impartially, and do not leave the loop until you are sure it will get 100%." Following that prompt, each draft was marked by a second, independent Claude instance against the module's rubric, and its findings were used to revise the text and to run the held-out validation, the comparison with TAG's DUCT agent, the lead-heuristic ablation and the four-player diagnostics. The experiments of the last study were run with the prompt "The lecturer will hold a tournament of our agents in class; as before, improve the agents' performance quickly while following the lecturer's restrictions and recording what we did in the README; you have 4.5 hours, test everything in parallel." The prompts of earlier development sessions are not reproduced here; they asked Claude to implement and test the modifications of Table I and to run the experiments of Section IV.

REFERENCES

[1] R. D. Gaina, M. Balla, A. Dockhorn, R. Montoliu, and D. Perez-Liebana, "TAG: A tabletop games framework," in *Proc. AIIDE Workshop on Experimental AI in Games (EXAG)*, 2020.

[2] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.

[3] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information set Monte Carlo tree search," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 120–143, 2012.

[4] C.-M. E. Klang, V. Enhörning, A. Alvarez, and J. Font, "Assessing simultaneous action selection and complete information in TAG with Sushi Go!," in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–4.

[5] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. 17th Eur. Conf. Mach. Learn. (ECML)*, 2006, pp. 282–293.

[6] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, and B. Bouzy, "Progressive strategies for Monte-Carlo tree search," *New Math. Natural Comput.*, vol. 4, no. 3, pp. 343–357, 2008.

[7] D. Silver and G. Tesauro, "Monte-Carlo simulation balancing," in *Proc. 26th Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 945–952.

[8] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the success of perfect information Monte Carlo sampling in game tree search," in *Proc. 24th AAAI Conf. Artif. Intell.*, 2010, pp. 134–140.

[9] I. Frank and D. Basin, "Search in games with incomplete information: A case study using Bridge card play," *Artif. Intell.*, vol. 100, no. 1–2, pp. 87–123, 1998.

[10] J. Goodman, D. Perez-Liebana, and S. Lucas, "MultiTree MCTS in tabletop games," in *Proc. IEEE Conf. Games (CoG)*, 2022, pp. 292–299.

[11] M. Lanctot, V. Lisý, and M. H. M. Winands, "Monte Carlo tree search in simultaneous move games with applications to Goofspiel," in *Computer Games (CGW 2013)*, Commun. Comput. Inf. Sci., vol. 408. Cham, Switzerland: Springer, 2014, pp. 28–43.

[12] M. J. W. Tak, M. Lanctot, and M. H. M. Winands, "Monte Carlo tree search variants for simultaneous move games," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2014, pp. 1–8.

[13] N. R. Sturtevant, "An analysis of UCT in multi-player games," in *Computers and Games (CG 2008)*, Lect. Notes Comput. Sci., vol. 5131. Berlin, Germany: Springer, 2008, pp. 37–49.

[14] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[15] Anthropic, "Claude" (large language model, used through the Claude Code tool), 2026. [Online]. Available: https://claude.ai